SOURCE CODE:

```
import java.util.Scanner;
interface Shape{
        double area();
        double perimeter();
}
class Rectangle implements Shape{
        double length;
        double breadth;
        Rectangle(double length, double breadth){
                this.length=length;
                this.breadth = breadth;
        }
        public double area(){
                return (length*breadth);
        }
        public double perimeter(){
                return 2*(length+breadth);
        }
}
class Circle implements Shape{
      double radius;
      Circle(double radius){
            this.radius = radius;
      }
      public double area(){
            return 3.14*radius*radius;
      }
      public double perimeter(){
```

Experiment No:19 Lab Cycle 3 Date: 16-03-2026

INTERFACE

AIM:

Create an interface having prototypes of functions area() and perimeter(). Create two classes Circle and Rectangle which implements the above interface. Create a menu driven program to find area and perimeter of objects.

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import Scanner class for input.
4. Define an interface Shape with methods area() and perimeter().
5. Create class Rectangle implementing Shape.
6. Define variables length and breadth.
7. Implement area() and perimeter() methods.
8. Create class Circle implementing Shape.
9. Define variable radius.
10. Implement area() and perimeter() methods.
11. In main(), create Scanner object.
12. Display menu options (Rectangle, Circle, Exit).
13. Read user choice.
14. If choice is Rectangle, read length and breadth.
15. Create Rectangle object.
16. Display area and perimeter.
17. If choice is Circle, read radius.
18. Create Circle object.
19. Display area and perimeter.
20. If choice is Exit, terminate program.
21. Repeat until user exits.
22. End program execution.
23. Stop

```
            return 3.14*radius*2;
        }
}
class InterfaceExample{
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                while(true){
                        System.out.println("\nMenu:");
                        System.out.println("1. Calculate Rectangle Area and
Perimeter");
                        System.out.println("2. Calculate Circle Area and Perimeter");
                        System.out.println("3. Exit");
                        System.out.print("Enter your choice: ");
                        int choice = sc.nextInt();
                        switch(choice){
                                case 1:
                                        System.out.print("Enter Length of Rectangle: ");
                                        double l = sc.nextDouble();
                                        System.out.print("Enter Breadth of Rectangle:
");
                                        double b = sc.nextDouble();
                                        Rectangle re = new Rectangle(l, b);
                                        System.out.println("Area of Rectangle: " +
re.area());
                                        System.out.println("Perimeter of Rectangle: " +
re.perimeter());
                                        break;
                                case 2:
                                        System.out.print("Enter Radius of Circle: ");
                                        double r = sc.nextDouble();
```

```
                    Circle ci = new Circle(r);
                    System.out.println("Area of Circle: " +
ci.area());
                    System.out.println("Perimeter of Circle: " +
ci.perimeter());
                    break;
                case 3:
                    System.out.println("Exiting...");
                    sc.close();
                    return;
                default:
                    System.out.println("Invalid choice. Please try
again.");
            }
        }
    }
}
```

OUTPUT:

```
Menu:
1. Calculate Rectangle Area and Perimeter
2. Calculate Circle Area and Perimeter
3. Exit
Enter your choice: 1
Enter Length of Rectangle: 5
Enter Breadth of Rectangle: 3
Area of Rectangle: 15.0
Perimeter of Rectangle: 16.0

Menu:
1. Calculate Rectangle Area and Perimeter
2. Calculate Circle Area and Perimeter
3. Exit
Enter your choice: 2
Enter Radius of Circle: 2
Area of Circle: 12.56
Perimeter of Circle: 12.56

Menu:
1. Calculate Rectangle Area and Perimeter
2. Calculate Circle Area and Perimeter
3. Exit
Enter your choice: 3
Exiting...
```

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
class ExceptionExample{
        public static void main(String args[]){
                try{
                        int a = 1/0;
                }catch(ArithmeticException e){
                        System.out.println("Arithmetic Exception : "+e);
                }

                try{
                int[] a = new int[4];
                        int b = a[10];
            }catch(ArrayIndexOutOfBoundsException e){
                System.out.println("Array Exception : "+e);
            }

        }
}
```

OUTPUT:

Arithmetic Exception : java.lang.ArithmeticException: / by zero

Array Exception : java.lang.ArrayIndexOutOfBoundsException: Index 10 out of bounds for length 4

Experiment No:20 Lab Cycle 4 Date: 16-03-2026

EXCEPTION CLASS

AIM:

Write a program that demonstrates exception handling mechanisms.

a) Arithmetic Exception.

b) Array Index Out Of Bounds Exception.

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Create main method.
4. Use try block for arithmetic operation.
5. Perform division by zero.
6. Catch ArithmeticException.
7. Display error message.
8. Use another try block.
9. Declare array of fixed size.
10. Access invalid index.
11. Catch ArrayIndexOutOfBoundsException.
12. Display error message.
13. End program execution.
14. Stop

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.util.Scanner;
class NegativeNumberException extends Exception{
	NegativeNumberException(String msg){
		super(msg);
	}
}
class UserException{
	public static void main(String args[]){
		Scanner sc = new Scanner(System.in);
		System.out.println("Enter total number of elements : ");
		int n = sc.nextInt();
		int sum = 0;
		int count = 0;
		while(count < n){
			try{
				System.out.println("Enter number " + (count + 1) + ":
");
				int a = sc.nextInt();
				if(a < 0){
					throw new NegativeNumberException("Only
positive numbers accepted");
				}
				sum += a;
				count++;
			}catch(NegativeNumberException e){
				System.out.println("Exception: " + e.getMessage());
			}
		}
		double average = (double) sum / n;
```

Experiment No:21 Lab Cycle 4 Date: 16-03-2026

USER DEFINED EXCEPTION

AIM:

Find the average of N Positive integers , raising a user defined exception for each negative input.

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import Scanner class.
4. Define custom exception class.
5. Create main method.
6. Read value of N.
7. Initialize sum = 0 and count = 0.
8. Repeat while count < N.
9. Read input number.
10. Check if number is negative.
11. If negative, throw custom exception.
12. Catch exception and display message.
13. If valid, add to sum.
14. Increment count.
15. Repeat loop.
16. Calculate average = sum / N.
17. Display average.
18. End program execution.
19. Stop

```
                System.out.println("Average is " + average);
        }
}
```

OUTPUT:

```
Enter total number of elements :
3
Enter number 1:
5
Enter number 2:
-2
Exception: Only positive numbers accepted
Enter number 2:
10
Enter number 3:
15
Average is 10.0
```

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.util.Scanner;
class Table5 extends Thread{
        int n;
        Table5(int n){
                this.n = n;
        }
        public void run(){
                for(int i=1;i<=n;i++)
                        System.out.println(i+"*5 = "+(i*5));
        }
}
class PrimeN extends Thread{
    int n;
    PrimeN(int n){
        this.n = n;
    }
    public void run(){
        for(int i=2;i<=n;i++){
            int f = 0;
            for(int j=2;j<=i/2;j++){
                if(i%j == 0){
                    f = 1;
                    break;
                }
            }
            if(f==0)
                System.out.println("prime : "+i);
        }
```

Experiment No:22 Lab Cycle 4 Date: 23-03-2026

THREAD CLASS

AIM:

Define 2 classes , one for generating a multiplication table of 5 and other for displaying first N prime numbers. Implement using threads. (Thread class).

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import Scanner class.
4. Create a class Table5 extending Thread.
5. Define run() method to generate multiplication table of 5.
6. Create another class PrimeN extending Thread.
7. Define run() method to display prime numbers up to N.
8. In main(), create Scanner object.
9. Read value of N.
10. Create objects of Table5 and PrimeN classes.
11. Start both threads using start() method.
12. Execute both threads simultaneously.
13. End program execution.
14. Stop

```
    }
}
class ThreadExample{
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                System.out.println("Enter N : ");
                int n = sc.nextInt();
                Table5 t1 = new Table5(n);
                PrimeN p1 = new PrimeN(n);
                t1.start();
                p1.start();
        }
}
```

OUTPUT:

```
Enter N :
5
1*5 = 5
prime : 2
2*5 = 10
prime : 3
3*5 = 15
prime : 5
4*5 = 20
5*5 = 25
```

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.util.Scanner;
class Fibonacci implements Runnable{
        int n;
        Fibonacci(int n){
                this.n = n;
        }
        int a = 0;
      int b = 1;
        public void run(){
                for (int i = 0; a<=n; i++) {
                        System.out.println("Fibonacci : "+a);
                        int next = a + b;
                        a = b;
                        b = next;

                }
        }
}
class EvenNumbers implements Runnable{
   int n;

   EvenNumbers(int n){
      this.n = n;
   }
   public void run(){
         for(int i = 1; i<=n; i++)
         if(i%2 == 0)
               System.out.println("Even : "+i);
```

Experiment No:23 Lab Cycle 4 Date: 23-03-2026

RUNNABLE INTERFACE

AIM:

Define 2 classes, one for generating Fibonacci numbers and other for displaying even numbers in a given range . Implement using threads. (Runnable Interface).

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import Scanner class.
4. Create a class Fibonacci implementing Runnable.
5. Define run() method to generate Fibonacci series.
6. Create another class EvenNumbers implementing Runnable.
7. Define run() method to display even numbers up to N.
8. In main(), create Scanner object.
9. Read value of N.
10. Create Thread objects using Runnable classes.
11. Start threads using start() method.
12. Execute both threads simultaneously.
13. End program execution.
14. Stop

```
    }
}
class RunnableExample{
        public static void main(String args[]){
                Scanner sc = new Scanner(System.in);
                System.out.println("Enter N : ");
                int n = sc.nextInt();
                Thread t1 = new Thread(new Fibonacci(n));
                Thread t2 = new Thread(new EvenNumbers(n));
                t1.start();
                t2.start();
        }
}
```

OUTPUT:

```
Enter N :
10
Fibonacci : 0
Even : 2
Fibonacci : 1
Even : 4
Fibonacci : 1
Even : 6
Fibonacci : 2
Even : 8
Fibonacci : 3
Even : 10
Fibonacci : 5
Fibonacci : 8
```

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.awt.*;
import java.awt.event.*;
public class Pgm24 extends Frame implements ActionListener {
   TextField t1, t2, t3, result;
   Button b;
   Pgm24(){
      Label l1 = new Label("Enter first number:");
      Label l2 = new Label("Enter second number:");
      Label l3 = new Label("Enter third number:");
      Label l4 = new Label("Largest number:");
      t1 = new TextField();
      t2 = new TextField();
      t3 = new TextField();
      result = new TextField();
      result.setEditable(false);
      b = new Button("Find Largest");
      b.addActionListener(this);
      setLayout(new GridLayout(5, 2));
      add(l1); add(t1);
      add(l2); add(t2);
      add(l3); add(t3);
      add(l4); add(result);
      add(b);
      setTitle("Largest of 3 Numbers");
      setSize(300, 300);
      setVisible(true);
      addWindowListener(new WindowAdapter() {
         public void windowClosing(WindowEvent we) {
```

Experiment No:24 Lab Cycle 5 Date: 23-02-2026

ABSTRACT WINDOW TOOLKIT – MAXIMUM OF THREE NUMBERS

AIM:

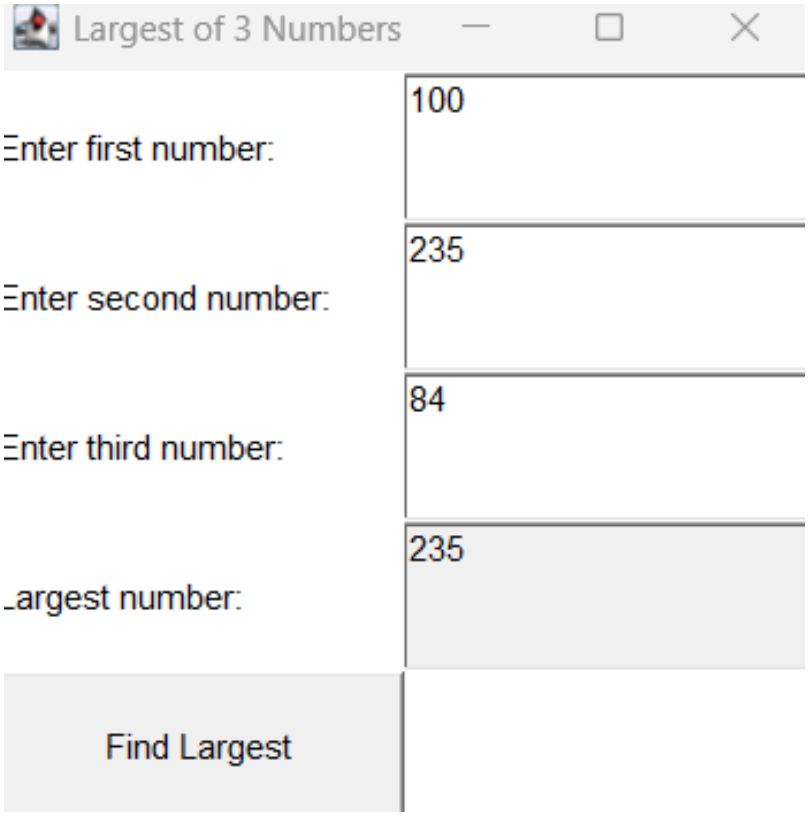
Program to find maximum of three numbers using AWT. (Abstract Window Toolkit).

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import AWT and event packages.
4. Create a class extending Frame.
5. Create labels, text fields and button.
6. Set layout and add components.
7. Add ActionListener to button.
8. Read input values from text fields.
9. Convert inputs to integers.
10. Compare three numbers.
11. Find largest number.
12. Display result in text field.
13. Handle invalid input using exception.
14. Add window closing event.
15. Display frame.
16. End program execution.
17. Stop

```
                dispose();
            }
        });
    }
    public void actionPerformed(ActionEvent e) {
        try {
            int a = Integer.parseInt(t1.getText());
            int b = Integer.parseInt(t2.getText());
            int c = Integer.parseInt(t3.getText());
            int largest;
            if (a >= b && a >= c)
                largest = a;
            else if (b >= a && b >= c)
                largest = b;
            else
                largest = c;
            result.setText(String.valueOf(largest));
        } catch (Exception ex) {
            result.setText("Invalid input");
        }
    }
    public static void main(String[] args) {
        new Pgm24();
    }
}
```

OUTPUT:


Largest of 3 Numbers
Enter first number:
100
Enter second number:
235
Enter third number:
84
Largest number:
235
Find Largest

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.awt.*;
import java.awt.event.*;
public class Pgm25 extends Frame implements ActionListener {
   TextField t1, t2, result;
   Button add, sub, mul, div;
   Pgm25() {
      Label l1 = new Label("Enter first number:");
      Label l2 = new Label("Enter second number:");
      Label l3 = new Label("Answer:");
      t1 = new TextField();
      t2 = new TextField();
      result = new TextField();
      result.setEditable(false);
      add = new Button("+");
      sub = new Button("-");
      mul = new Button("*");
      div = new Button("/");
      add.addActionListener(this);
      sub.addActionListener(this);
      mul.addActionListener(this);
      div.addActionListener(this);
      setLayout(new GridLayout(5, 2));
      add(l1); add(t1);
      add(l2); add(t2);
      add(add); add(sub);
      add(mul); add(div);
      add(l3); add(result);
      setTitle("Calculator");
```

Experiment No:25 Lab Cycle 5 Date: 30-03-2026

ABSTRACT WINDOW TOOLKIT – SIMPLE CALCULATOR

AIM:

Write a program to implement a simple calculator using AWT components.

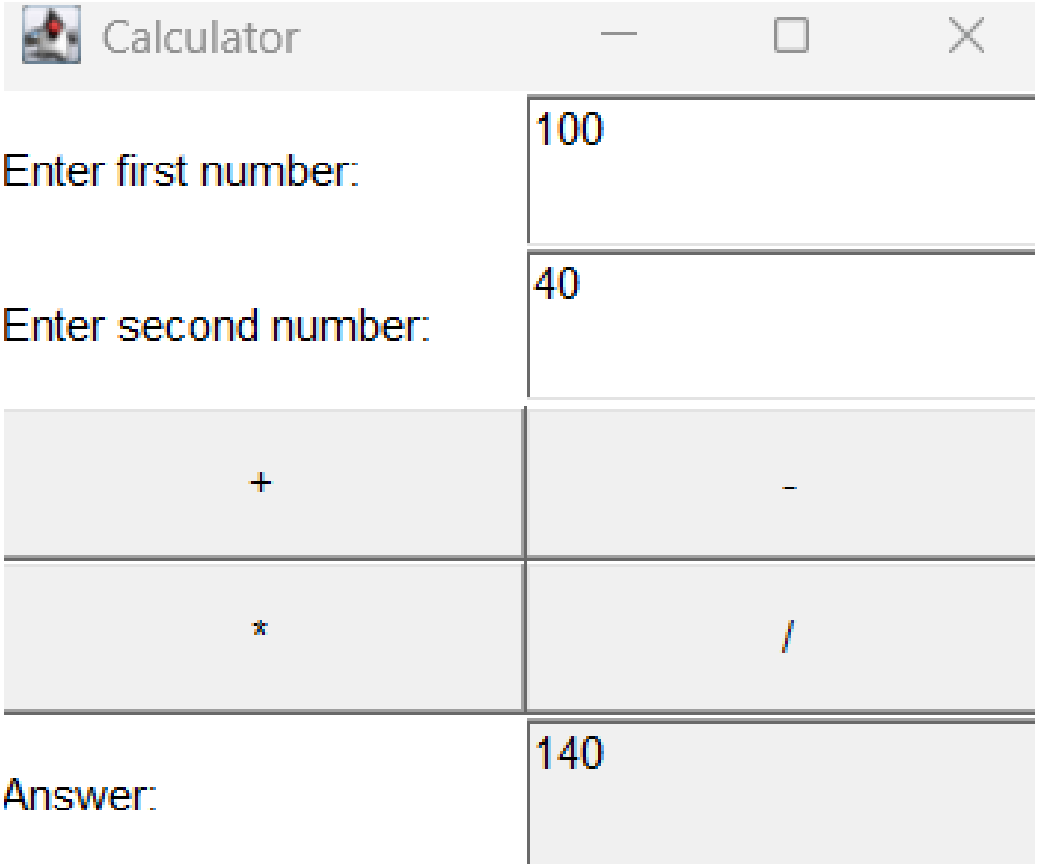
ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import AWT and event packages.
4. Create a class extending Frame.
5. Create labels, text fields and buttons for operations.
6. Set layout and add components.
7. Add ActionListener to all buttons.
8. Read input values from text fields.
9. Convert inputs to integers.
10. Check which button is pressed.
11. Perform corresponding operation (+, -, *, /).
12. Handle division by zero.
13. Display result in text field.
14. Handle invalid input using exception.
15. Add window closing event.
16. Display frame.
17. End program execution.
18. Stop

```
    setSize(300, 250);
    setVisible(true);
    addWindowListener(new WindowAdapter() {
        public void windowClosing(WindowEvent we) {
            dispose();
        }
    });
}
public void actionPerformed(ActionEvent e) {
    try {
        int a = Integer.parseInt(t1.getText());
        int b = Integer.parseInt(t2.getText());
        int ans = 0;
        if (e.getSource() == add)
            ans = a + b;
        else if (e.getSource() == sub)
            ans = a - b;
        else if (e.getSource() == mul)
            ans = a * b;
        else if (e.getSource() == div){
                if (b == 0) {
                        result.setText("Cannot divide");
                        return;
                }
            ans = a / b;
            }
        result.setText(String.valueOf(ans));
    } catch (Exception ex) {
```

```
            result.setText("Error");
        }
    }
    public static void main(String[] args) {
        new Pgm25();
    }
}
```

OUTPUT:



RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.io.*;
public class FileEdit {
    public static void main(String[] args) {
        try {
            FileWriter fw = new FileWriter("test.txt");
            fw.write("Hello, this is my first file.\n");
            fw.write("Learning Java file handling.");
            fw.close();
            System.out.println("Data written to file.");
            FileReader fr = new FileReader("test.txt");
            BufferedReader br = new BufferedReader(fr);
            String line;
            System.out.println("\nReading from file:");
            while ((line = br.readLine()) != null) {
                System.out.println(line);
            }
            br.close();
        } catch (IOException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

OUTPUT:

```
Data written to file.

Reading from file:
Hello, this is my first file.
Learning Java file handling.
```

Experiment No:26 Lab Cycle 6 Date: 30-03-2026

FILE MANIPULATION

AIM:

Write a program to write to a file, then read from the file and display the contents on the console. (File Manipulation in Java).

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import java.io package.
4. Create FileWriter object.
5. Write data into file.
6. Close FileWriter.
7. Display confirmation message.
8. Create FileReader object.
9. Wrap with BufferedReader.
10. Read file line by line.
11. Display each line.
12. Close BufferedReader.
13. Handle IOException.
14. End program execution.
15. Stop

RESULT:

Program is executed successfully and output is verified.

SOURCE CODE:

```
import java.io.*;
public class FileCopy {
    public static void main(String[] args) {
        try {
            FileInputStream fin = new FileInputStream("source.txt");
            FileOutputStream fout = new FileOutputStream("destination.txt");
            int data;
            while ((data = fin.read()) != -1) {
                fout.write(data);
            }
            fin.close();
            fout.close();
            System.out.println("File copied successfully.");
        } catch (IOException e) {
            System.out.println("Error: " + e);
        }
    }
}
```

OUTPUT:

File copied successfully.

Experiment No:27 Lab Cycle 6 Date: 30-03-2026

FILE MANIPULATION

AIM:

Write a program to copy one file to another.

ALGORITHM:

1. Start
2. Begin execution of the program.
3. Import java.io package.
4. Create FileInputStream for source file.
5. Create FileOutputStream for destination file.
6. Read data byte by byte from source file.
7. Write data into destination file.
8. Repeat until end of file.
9. Close both file streams.
10. Display success message.
11. Handle IOException.
12. End program execution.
13. Stop

RESULT:

Program is executed successfully and output is verified.